

TRANS-01 Project 5 – Annotated Code Document

Civil Engineering Analysis II | Spring 2026

Logan Baardson, Arturo Garcia, Nay Law Htoo

Client: Nebraska Department of Transportation (NDOT)

Section 1 - Package Imports

Code	Annotation/Explanation
<code>import pandas as pd</code>	This loads the pandas library. It is used throughout for reading CSV, filtering rows, grouping by agent type, and simple computing of summary statistics.
<code>import numpy as np</code>	Loads numpy for numerical support. Aliased as 'np'. Acts as a dependency for pandas and matplotlib calculations behind the scenes.
<code>import matplotlib.pyplot as plt</code>	Loads matplotlib's pyplot module as 'plt'. This is the core engine for drawing all plots: boxplots, line charts, bar charts, and the trajectory map.
<code>import seaborn as sns</code>	Loads seaborn as 'sns'. Adds a polished visual theme on top of matplotlib and provides the color palettes used in several plots.
<code>sns.set_style('whitegrid')</code>	Applies seaborn's 'whitegrid' theme globally. This adds subtle horizontal gridlines to every figure and gives plots a clean white background.
<code>sns.set_palette('deep')</code>	Sets the default color palette to 'deep' for all seaborn-driven plots. Ensures consistent, readable colors across figures.

Section 2 – Loading and Previewing the Dataset

Code	Annotation/Explanation
------	------------------------

<pre>tgsim_data = pd.read_csv('Cleaned_TGSIM_Foggy_Bottom.csv')</pre>	Reads the CSV file from the working directory into a pandas DataFrame. The file must be in the same folder as the notebook.
<pre>tgsim_data = pd.DataFrame(tgsim_data)</pre>	Explicitly converts the loaded object into a pandas DataFrame structure. Good practice to confirm data type before analysis.
<pre>tgsim_data['agent_type_label'] = tgsim_data['agent_type_label'].str.strip()</pre>	Removes invisible leading/trailing whitespace from agent labels. The raw CSV had a leading space on some entries (e.g., ' Trucks') that would silently break grouping and filtering without this fix.
<pre>tgsim_data.columns</pre>	Prints all column names. Used to verify the file loaded with the expected structure before running any analysis.
<pre>tgsim_data.head(5)</pre>	Displays the first 5 rows. A quick visual check that values, data types, and column names look correct after loading.
<pre>tgsim_data.describe()</pre>	Generates summary statistics (count, mean, std, min, max, quartiles) for every numeric column. Useful for checking data ranges and spotting outliers or errors.

<code>print(tgsim_data['agent_type_label'].value_counts())</code>	Counts how many rows (observations) exist per agent type. Key finding: only 1 Automated Vehicle exists among 564 total agents as this limits how broadly AV conclusions can be generalized.
---	---

Section 3 – Summary Statistics by Agent Type

Code	Annotation/Explanation
<code>tgsim_grouped = tgsim_data.groupby(['agent_type_label'])</code>	Groups all rows that share the same <code>agent_type_label</code> . All subsequent <code>.agg()</code> calculations are applied independently within each group.
<code>summary_table = tgsim_grouped.agg(...)</code>	Applies multiple aggregation functions in a single step. Each named argument defines a new output column: the source column and statistic to compute (e.g., mean, std, count, nunique).
<code>num_observations = ('speed_magnitude', 'count')</code>	Counts the total number of data rows for each agent type. Represents how many 0.1-second timesteps were recorded, not how many individual agents.
<code>num_agents = ('id', 'nunique')</code>	Counts the number of unique agent IDs in each group. Tells us how many individual vehicles or pedestrians of that type were tracked.
<code>mean_speed_ms = ('speed_magnitude', 'mean')</code>	Calculates the average speed (in m/s) across all observations for each agent type.

<code>std_speed_ms = ('speed_magnitude', 'std')</code>	Calculates the standard deviation of speed. Lower std = more consistent/predictable speed behavior. This is a key metric for evaluating whether AVs are more predictable than other agents.
<code>summary_table.sort_values('mean_speed_ms', ascending=False)</code>	Sorts the summary table by mean speed in descending order so the fastest agent types appear at the top. ascending=False means highest values come first.

Section 4 – Speed and Acceleration Boxplots

Code	Annotation/Explanation
<code>plt.figure(figsize=(14, 6))</code>	Creates a blank figure canvas 14 inches wide and 6 inches tall. The wider size gives enough space for 8 agent type labels on the x-axis without overlapping.
<code>tgsm_data.boxplot(column='speed_magnitude', by='agent_type_label', rot=30)</code>	Draws one box per agent type, grouped by the 'by' column. Each box shows the median, IQR, and outliers for speed_magnitude. rot=30 tilts x-axis labels 30 degrees to prevent overlap.
<code>plt.xlabel / plt.ylabel</code>	Sets descriptive axis labels with fontsize=12 to ensure they are clearly readable.
<code>plt.title(...)</code>	Sets the figure title. Note: when using pandas .boxplot() with a 'by' argument, pandas auto-generates a secondary title so plt.title() overrides it with the desired label.

<code>plt.tight_layout()</code>	Automatically adjusts subplot spacing so that axis labels, tick marks, and titles don't get clipped at the figure edges.
<code>plt.show()</code>	Renders and displays the completed figure inline in the notebook. Always called after all formatting commands are in place.
<code>column='accel_magnitude'</code>	The second boxplot uses <code>accel_magnitude</code> instead of <code>speed_magnitude</code> . Negative values indicate deceleration; positive values indicate acceleration. The spread shows how consistently each agent type changes speed.

Section 5 – Speed Over Time: Avs vs. Passenger Cars

Code	Annotation/Explanation
<code>tgsim_data[tgsim_data['agent_type_label'] == 'Automated Vehicles']</code>	Boolean filter: creates a True/False mask for each row based on whether the <code>agent_type_label</code> matches. Wrapping the DataFrame in <code>[...]</code> returns only the rows where the condition is True.
<code>.groupby('time')['speed_magnitude'].mean()</code>	After filtering, groups remaining rows by their timestamp and computes the average speed at each moment. Since there is only 1 AV, this is simply that vehicle's speed at each timestep with no averaging.
<code>plt.plot(av_speed_over_time.index, av_speed_over_time.values, ...)</code>	Plots the AV line. <code>.index</code> contains the time values (x-axis) and <code>.values</code> contains the mean speed values (y-axis). <code>label=</code> registers the line in the legend.

<code>plt.plot(car_speed_over_time.index, car_speed_over_time.values, ...)</code>	Plots the Passenger Car line in red on the same axes. Both <code>plt.plot()</code> calls share the same figure, so the lines are automatically overlaid for direct comparison.
<code>plt.legend(fontsize=11)</code>	Draws the legend using the label= strings passed to each <code>plt.plot()</code> call. <code>fontsize=11</code> keeps the legend readable without dominating the figure.
<code>plt.grid(True, alpha=1)</code>	Adds fully opaque gridlines behind the data lines. Helps trace specific time values across the chart when interpreting speed changes at particular moments.

Section 6 – Function 1: `classify_behavior(accel)`

Code	Annotation/Explanation
<code>def classify_behavior(accel):</code>	The <code>def</code> keyword defines a reusable function named <code>classify_behavior</code> . It accepts one argument, <code>accel</code> , which will be passed the acceleration value from each row when <code>.apply()</code> calls it.
<code>if accel <= -2.0: return 'Hard Deceleration'</code>	First condition: any acceleration at or below -2.0 m/s^2 is classified as Hard Deceleration. Python checks conditions top to bottom and stops at the first True match.
<code>elif -2.0 < accel <= -0.5: return 'Mild Deceleration'</code>	Second condition: catches moderate

	deceleration between -2.0 and -0.5 m/s ² . The double inequality checks both bounds in a single expression.
<code>elif -0.5 < accel <= 0.5: return 'Steady'</code>	Third condition: values near zero represent roughly constant speed. The ± 0.5 m/s ² window accounts for minor real-world fluctuations that aren't meaningful acceleration changes.
<code>elif 0.5 < accel <= 2.0: return 'Mild Acceleration'</code>	Fourth condition: moderate positive acceleration up to 2.0 m/s ² .
<code>else: return 'Hard Acceleration'</code>	Catch-all for any acceleration above 2.0 m/s ² . The else handles all remaining cases without needing to specify an upper bound.
<code>tgsim_data['behavior_label'] = tgsim_data['accel_magnitude'].apply(classify_behavior)</code>	Calls <code>classify_behavior</code> once per row, passing that row's <code>accel_magnitude</code> as the argument. The return value (a string label) is stored in a new column called <code>behavior_label</code> .
BIAS NOTE: threshold values	The cutoffs (-2.0, -0.5, 0.5, 2.0 m/s ²) were set using engineering judgment, not a published standard. Changing them would shift how events are

	classified and alter which agent types appear more or less aggressive. This should be disclosed to the client.
<pre>behavior_summary = tgsim_data.groupby(['agent_type_label', 'behavior_label']).size().reset_index(name='count')</pre>	Groups rows by both agent type AND behavior label simultaneously, then counts how many rows fall into each combination. <code>.reset_index()</code> converts the multi-index result back into a flat DataFrame with a 'count' column.
<pre>behavior_pivot = behavior_summary.pivot(index='agent_type_label', columns='behavior_label', values='count').fillna(0)</pre>	Reshapes the long-format summary into a wide table: each behavior category becomes its own column. <code>.fillna(0)</code> replaces NaN with 0 for any agent type that had zero observations in a particular category.
<pre>behavior_order = ['Hard Deceleration', 'Mild Deceleration', 'Steady', 'Mild Acceleration', 'Hard Acceleration']</pre>	Defines the left-to-right display order of bars. Without this, pandas would sort columns alphabetically, which would place Hard Acceleration before Hard Deceleration and break the logical flow.
<pre>[col for col in behavior_order if col in behavior_pivot.columns]</pre>	List comprehension that filters <code>behavior_order</code> to only

	include categories that actually exist in the pivot table. Prevents a KeyError if a category has zero observations across all agents.
<code>behavior_plot_data.plot(kind='bar', color=behavior_colors, edgecolor='white', width=0.75)</code>	Draws a grouped bar chart where each agent type gets a cluster of colored bars so that it gets one per behavior category. edgecolor='white' adds thin white gaps between bars for readability.

Section 7 – Function 2: `count_decel_events(df, threshold)`

Code	Annotation/Explanation
<code>def count_decel_events(df, threshold=-2.0):</code>	Defines the function with two parameters: <code>df</code> (the DataFrame to analyze) and <code>threshold</code> (the cutoff for 'hard' deceleration). <code>threshold=-2.0</code> is a default value as the function works without specifying it, but can be overridden for sensitivity testing.
<code>hard_decel_df = df[df['accel_magnitude'] <= threshold]</code>	Filters the DataFrame to

	keep only rows where acceleration is at or below the threshold. This isolates the timesteps that qualify as hard deceleration events.
<pre>decel_summary = hard_decel_df.groupby('agent_type_label').agg(total_decel_ events=('id','count'), num_agents=('id','nunique'))</pre>	Groups the filtered data by agent type and computes two statistics: total_decel_events (total row count = total events) and num_agents (unique agent IDs = how many individuals experienced them).
<pre>decel_summary['avg_events_per_agent'] = (decel_summary['total_decel_events'] / decel_summary['num_agents']).round(2)</pre>	Divides total events by unique agents to normalize for group size. Without this, Passenger Cars (157 agents) would always rank highest over AVs (1 agent). The normalized rate enables a fair per-agent comparison.

	.round(2) keeps output clean.
return decel_summary.sort_values('avg_events_per_agent', ascending=False)	Returns the completed summary table sorted from highest to lowest avg events per agent. The function call result is stored in decel_results for subsequent plotting.
decel_results = count_decel_events(tgsim_data, threshold=-2.0)	Calls the function with the default threshold. Storing the output in decel_results makes it available for the bar chart in the next cell.
sns.color_palette('deep', len(decel_results))	Generates one unique color per bar by requesting len(decel_results) colors from the 'deep' palette. This automatically adjusts if the number of agent types in the result changes.

Section 8 – Function 3: plot_agent_trajectories(df, agent_types, polygons)

Code	Annotation/Explanation
------	------------------------

<code>color_map = {'Automated Vehicles': 'blue', ...}</code>	A dictionary mapping each agent type label to a specific color string. Used to look up the correct color when plotting each agent's path.
<code>color = color_map.get(agent_type, 'black')</code>	<code>.get()</code> looks up the agent type in <code>color_map</code> . If a type is not found (unexpected label), it returns 'black' as a safe fallback instead of crashing with a <code>KeyError</code> .
<code>fig, ax = plt.subplots(figsize=(14, 10))</code>	Creates a figure with a single Axes object (<code>ax</code>). Using <code>ax.plot()</code> directly (instead of <code>plt.plot()</code>) is better practice inside functions to avoid accidentally drawing on a previously created figure.
<code>coords_closed = coords + [coords[0]]</code>	Polygons are open lists of vertices. Appending the first coordinate to the end 'closes' the shape so the outline connects back to the starting point, completing the polygon boundary.
<code>x, y = zip(*coords_closed)</code>	<code>zip(*...)</code> transposes a list of (x,y) tuples into two separate sequences: all x-values and all y-values. This is the format matplotlib expects for plotting coordinates.
<code>ax.fill(x, y, alpha=0.12, color='gray')</code>	Draws a semi-transparent gray fill inside each lane polygon. <code>alpha=0.12</code> keeps the fill subtle so trajectory lines remain clearly visible on top.
<code>ax.plot(x, y, linestyle='--', linewidth=0.8, color='gray', alpha=0.6)</code>	Draws the dashed polygon outline on top of the fill. Both fill and outline together give the lanes visual presence without

	overwhelming the trajectory data.
<code>path = agent_subset[agent_subset['id'] == agent_id].sort_values('time')</code>	Isolates all rows for one specific agent and sorts them chronologically. Without <code>.sort_values('time')</code> , the path line could jump backwards in time, creating false visual crossing patterns.
<code>ax.plot(path['xloc_kf'], path['yloc_kf'], color=color, linewidth=0.9, alpha=0.5)</code>	Draws the agent's trajectory as a line connecting its (x,y) positions in time order. <code>alpha=0.5</code> makes overlapping paths partially transparent so all paths remain visible in dense areas.
<code>ax.plot([], [], color=color, linewidth=2.5, label=agent_type)</code>	Draws an invisible 'proxy' line (empty x and y) used only to register a legend entry. Without this, matplotlib would add one legend entry per individual agent path (hundreds of entries).
<code>bbox_to_anchor=(1.02, 1), loc='upper left'</code>	Places the legend just outside the right edge of the plot area. This prevents the legend from covering any part of the trajectory map.
<code>plt.tight_layout(rect=[0, 0, 0.85, 1])</code>	Tells <code>tight_layout</code> to use only 85% of the figure width for the axes, reserving 15% on the right for the external legend so it is not clipped.
<code>ax.set_aspect('equal')</code>	Forces the x and y axes to use the same scale. Since both axes represent meters, this ensures the intersection geometry appears at its true proportions and lanes are not stretched or squashed.

all_agent_types = tgsim_data['agent_type_label'].unique().tolist()	.unique() retrieves all distinct agent type strings. .tolist() converts the result to a plain Python list. Passing this to the function ensures every type in the data is plotted, automatically adapting if types are added or removed.
av_and_cars = ['Automated Vehicles', 'Passenger Cars (Non-automated)']	A manually defined list for the second call. Passing a smaller subset filters the plot to only those two agent types, producing a focused AV-vs-car comparison without modifying the function itself.
plot_agent_trajectories(tgsim_data, av_and_cars, polygons)	Second call reusing the same function with different inputs. This demonstrates the value of parameterizing agent_types — the function is flexible enough to serve both a full overview and a focused comparison.

Section 9 – Packages Used

Package	Purpose In This Analysis
pandas	Loading the CSV, cleaning labels, filtering rows, groupby aggregation, pivot tables, summary statistics
numpy	Numerical support and array operations; dependency for pandas and matplotlib calculations
matplotlib	Drawing all figures: boxplots, line charts, bar charts, and the agent trajectory map
seaborn	Visual theme (whitegrid), color palettes (deep), and color generation for bar charts